Jedinično testiranje u programskom jeziku Java





Autor: Goran Savić



Test code is just as important as production code. It is not a second-class citizen. It requires thought, design, and care. It must be kept as clean as production code.

(Clean Code: A Handbook of Agile Software Craftsmanship)

Robert C. Martin
(1952)

Jedinično testiranje u Javi

- Vrši se pisanjem Java programskog koda
- *Build-operate-check* šablon
  1. Kreiranje test podataka
     - U Javi je to najčešće instanciranje objekata
  2. Operacija nad podacima
     - U Javi je to poziv funkcionalnosti objekta čije ponašanje testiramo
  3. Provera podataka nakon operacije
     - U Javi je to poređenje stanja objekata sa očekivanim stanjem

F.I.R.S.T. pravila za pisanje testova

- Fast
  - Testovi trebaju biti takvi da se brzo izvršavaju
  - Ako se ne izvršavaju brzo, neće biti često pokretani, što će smanjiti kvalitet softvera
- Independent
  - Testovi ne treba da zavise jedan od drugog
  - Jedan test ne treba da postavlja preduslove za neki drugi test
  - Testovi trebaju moći da se izvršavaju nezavisno u proizvoljnom redosledu

F.I.R.S.T. pravila za pisanje testova

- Repeatable
  - Testovi trebaju moći ponovo da se izvrše u različitim okruženjima
  - U svakom okruženju u kojem se aplikacija razvija i koristi (produkcioni server, lokalni računar), test mora moći da se izvrši
- Self-Validating
  - Test treba da ima jedan *boolean* rezultat (da li je prošao ili ne)
  - Rezultat treba da se dobije izvršavanjem testa, a ne ručnim poređenjem ili subjektivnom procenom

F.I.R.S.T. pravila za pisanje testova

- Timely
  - Testove treba pisati blagovremeno
  - Test treba napisati **pre** koda koji će biti testiran

Radni okviri za testiranje u Javi

- Postoje radni okviri koji pružaju pomoć u
  - pisanju koda za testiranje
    - sadrže biblioteke specijalizovane za testiranje
  - izvršavanju testova
  - izveštavanju o testovima

- Najpopularniji radni okviri
  - JUnit
  - TestNG

JUnit

- JUnit je radni okvir za testiranje Java programa
- Poslednja verzija je JUnit 5
- Prema jednom istraživanju, JUnit je najčešće uključivana eksterna biblioteka u Java projekte

Asertacije

- Najvažniji deo testa je verifikacija dobijenog rezultata
  - Poređenje očekivane i stvarne vrednosti
- Vrši se asertacijama
- Asertacija je deo koda u kojem se zahteva da određeni izraz bude istinit
  - ako to nije slučaj, program prestaje da radi ili se dešava izuzetak
- Koristi se u testiranju pri utvrđivanju da li stvarna vrednost odgovara očekivanoj
  - ako to nije slučaj, test nije uspešan

JUnit asertacije

- JUnit predviđa različite funkcije za asertaciju
  - `assertTrue()`
  - `assertFalse()`
  - `assertEquals()`
  - `assertArrayEquals()`
  - `assertNull()`
  - `asertNotNull()`
  - `assertSame()`
  - `assertNotSame()`
  - `assertThat()`

JUnit asertacije

- Primer korišćenja asertacije

```
//instanciranje objekta cije ponasanje testiramo
ExampleUnit myUnit = new ExampleUnit();
//poziv testirane metode i preuzimanje stvarnog rezultata
String result = myUnit.concatenate("one", "two");
//poredjenje ocekivane i stvarne vrednosti
//ako vrednosti nisu jednake, test ne prolazi
assertEquals("onetwo", result);
```



- Primeri
  - `unit-testing`
    - `unitesting.junit`

Matchers

- JUnit pruža mogućnost definisanja asertacija i putem assertThat() metode
  - metoda prima Matcher izraze
- Primer

```
assertThat("that", is(equalTo("that")));
```



  - `unit-testing/unittesting.junit.AssertThatTest.java`
- Prednosti ovog načina asertacije
  - čitljivija sintaksa
  - asertacija se zadaje kroz subjekat, predikat i objekat
  - Matcher izrazi se mogu negirati i kombinovati konjukcijom ili disjunkcijom
  - čitljivije poruke o grešci kada test ne prolazi
- Hamcrest projekat sadrži veliki broj Matcher izraza
  - najznačajniji su uključeni u JUnit

AssertJ

- AssertJ pruža još fleksibilniji način definisanja asertacija koje koriste Matcher izraze
- Različite metode za asertaciju koje vraćaju novi objekat nad kojim se može vršiti asertacija
  - Na ovaj način je obezbeđeno ulančavanje metoda za asertaciju
  - Može se koristiti IDE *code-completion* kao pomoć za korišćenje asertacije
    - zavisno od konkretnog objekta, različite metode za asertaciju su dostupne



- Primer

```
assertThat("that").isEqualTo("that");
```

- `unit-testing/unittesting.assertj.AssertThatTest.java`

TestNG

- Drugi popularan radni okvir za testiranje Java programa
- Razvijen na osnovu JUnit da obezbedi naprednije upravljanje testiranjem u odnosu na JUnit
- JUnit verzije 4 je implementirao većinu funkcija koje je ranije samo TestNG obezbeđivao
- U nastavku su prikazane naprednije tehnike testiranja korišćenjem TestNG
  - većina toga može biti realizovana kroz JUnit u neznatno drugačijoj sintaksi

TestNG – Test Suite

- Testovi se mogu grupisati u Test Suite
  - U istom *suite* treba da se nađu logički srodni testovi
  - Ovi testovi se onda mogu izvršavati u grupi
- Test Suite se definiše kao XML fajl
- Sadrži niz testova koji se izvršavaju
- Za svaki test se navode klase čije @Test metode test sadrži

TestNG – Kategorizacija testova

- Test se može dodeliti određenoj grupi (kategoriji)
  - moguće je onda izvršiti samo testove iz jedne grupe
- Dodeljivanje testa u grupu
  - `@Test(groups = {"postgres-test"})`
  - jedan test može biti u više grupa
  - ako se anotacija stavi nad klasom onda svi testovi u toj klasi pripadaju grupi
- Izvršavanje određene grupe testova
  - u *test* suite se za test navodi na koje se grupe odnosi

TestNG – Test Suite i kategorizacija testova



- Primer kategorizacije
  - `unit-testing/unittesting.testng.groups.TestGroup.java`
  - `unit-testing/unittesting.testng.groups.TestPostgres.java`



- Primer suite
  - `unit-testing/postgres-suite.xml`
  - `unit-testing/database-suite.xml`

TestNG – Događaji

- Moguće je pri izvršavanju testova određene metode izvršiti na odgovarajućim mestima u životnom ciklusu izvršavanja testova
  - `@BeforeSuite, @AfterSuite`
    - metoda se izvršava pre ili posle izvršavanja *suite*
  - `@BeforeMethod, @AfterMethod`
    - metoda se izvršava pre ili posle svake `@Test` metode
  - `@BeforeTest, @AfterTest`
    - metoda se izvršava pre ili posle *test* taga u test *suite*
  - `@BeforeGroups, @AfterGroups`
    - metoda se izvršava pre ili posle izvršavanja određene grupe
  - `@BeforeClass, @AfterClass`
    - metoda se izvršava pre prve ili nakon poslednje `@Test` metode u klasi



- Primer
  - `unit-testing/all-suite.xml`
  - `unit-testing/unitestting.testng.beforeafter.*`

TestNG – očekivani izuzeci

- Test može da definiše da očekuje da se pri pozivu metode desi određeni izuzetak
  - test prolazi ako se takav izuzetak desi
  - ovo se koristi najčešće za negativne testove kada verifikujemo ponašanje na nevalidne ulaze
  - može se postaviti i da očekuje više izuzetaka
    - bilo koji izuzetak ako se desi, test prolazi



- Primer
  - `@Test(expectedExceptions = IndexOutOfBoundsException.class)`
  - `unit-testing/unittesting.testng.exceptions.TestException.java`

TestNG – Ograničenje vremena izvršavanja

- Moguće je definisati maksimalno vreme za koje test mora biti izvršen da bi bio uspešan
  - vreme se navodi u milisekundama
- Koristi se kod testiranja performansi



- Primer
  - `@Test(timeOut = 1000)`
  - `unit-testing/unittesting.testng.time.TestTimeout`

TestNG – Ignorisanje testova

- Za određene testove se može definisati da se uopšte ne izvršavaju



- Primer
  - `@Test(enabled = false)`
  - `unit-testing/unittesting.testng.ignore.TestIgnore`

TestNG - Zavisnosti

- Može se definisati da izvršavanje određenog testa zavisi od uspešnog izvršavanja drugih testova
  - test se ne izvršava ako zavisni testovi nisu prošli
- Zavisnost od pojedinačnih testova
  - `@Test(dependsOnMethods = { "test1", "test2"})`
- Zavisnost od cele grupe
  - `@Test(dependsOnGroups = {"group1","group2"})`



- Primer
  - `unit-testing/unittesting.testng.depend.*`

TestNG – Parametri testova

- Testovi mogu u toku izvršavanja da dobijaju parametre
- Ako se pošalje lista parametara, TestNG će kreirati više instanci testova, pri čemu će svaka instanca dobijati naredni iz liste
- Dva načina prosleđivanja parametara
- Iz XML fajla



  - Primer
    - `unit-testing/unittesting.testng.param.TestParamXML`
    - `unit-testing/unittesting.testng.param.parameter-test.xml`
- Iz programskog koda putem `@DataProvider` anotacije



  - Primer
    - `unit-testing/unittesting.testng.param.TestParamDataProvider`

Mocking

- Mocking mehanizam omogućuje simulaciju ponašanja objekata koje testirani objekat koristi
  - da bi se testirani objekat testirao u izolaciji važno je da referencirani objekti ne unose grešku
  - potrebno je simulirati da referencirani objekti uvek rade ispravno
- Umesto pravih objekata koje klasa referencira postavljaju se objekti dvojnici (*test doubles*) koji simuliraju ponašanje referenciranih objekata

Java Mock objekti

- Mockito 

  - Najpopularniji radni okvir za implementaciju *mocking* mehanizma u testiranju Java programa
- Omogućuje kreiranje objekta dvojnika
  - objekat ima isti interfejs kao originalni
  - metode se ne izvršavaju nad stvarnim objektom nego vraćaju simulirani rezultat koji mi odredimo
- *Mockito* sintaksno ne razlikuje *stub* od *mock* objekata
  - objekti se kreiraju metodom *mock()*
  - kontekst korišćenja pravi razliku između ova dva tipa

Mockito – Mock objekat



- Kreiranje *mock* objekta
  - statičkoj *mock()* metodi se prosleđuje klasa čiji lažni objekat kreiramo

```
LoginService ls = Mockito.mock(LoginService.class);
```

- Za metode *mock* objekta se mora definisati ponašanje
  - koji rezultat određena metoda vraća za određene izlaze
  - metode za koje se ne navede lažno ponašanje vraćaju *null*
  - u terminologiji testiranja, ovo odgovara *stub* objektu

```
//primer lazni login koji uvek uspeva
when(ls.login("admin", "admin")).
    thenReturn(true);
```

Mockito – Mock objekat



- Pri izvršavanju testa, *mock* objekat će se ponašati kako je specificirano

```
assertTrue(ls.login("admin", "admin"));
```

- Metoda mock objekta može da simulira izbacivanje određenog izuzetka za neke ulaze

```
doThrow(new UserNotFoundException()).
    when(ls).login("", "");
```



- Primeri
  - `unit-testing/unittesting.mock.*`

Mockito – Spy objekat



- Osim *mock* objekata *mockito* omogućuje i kreiranje *spy* objekata
- *Spy* objekat
  - objekat kod kojeg se pozivaju stvarne metode, osim onih za koje je eksplicitno navedeno simulirano ponašanje
  - kod *mock* objekta metode za koje se ne navede ponašanje vraćaju *null*, a kod *spy* objekta pozvaće se stvarna metoda

Mockito – Spy objekat



- Primer korišćenja spy objekta

```
//spy objekat za Java ArrayListu
ArrayList<int> list = new ArrayList<int>();
ArrayList spy = spy(list);
//ubacimo u listu vrednosti 5, 5, 5, 5
...
//lazno ponasanje za preuzimanje prvog elementa
doReturn(9).when(spy).get(0);
spy.get(0); //vraca 9
spy.get(1); //vraca 5 jer poziva stvarnu metodu
```



- Primeri
  - unit-testing/unittesting.mock.SpyTest

Mockito - Verifikacija



- *Mockito* omogućuje i tehnike *white-box* testiranja korišćenjem lažnih objekata
- *Mockito* obezbeđuje informacije koliko puta je metoda lažnog objekta pozivana
- Na ovaj način se osim stvarnog rezultata (kao kod *black-box* testiranja) može proveriti i da li testirani objekat referencira druge objekte na odgovarajući način
  - npr. ako smo putem *mock* objekta simulirali snimanje u bazu, možemo proveriti da li je metoda za snimanje pozvana tačno jednom
  - ako se *mock* objekat ovako koristi, to odgovara *mock* objektu u terminologiji testiranja

Mockito - Verifikacija



- Koristi se `verify()` metoda kojoj se prosleđuje
  - koji *mock* objekat verifikujemo
  - koju metodu tog objekta verifikujemo
  - broj očekivanih poziva te metode
- Broj očekivanih poziva se zadaje kao
  - konkretan broj
    - `times(3)`
  - implicitno bez navođenja
    - tada se smatra da se zahteva jedan poziv
  - nikad
    - `never()`
  - minimalan/maksimalan broj poziva
    - `atLeastOnce()`
    - `atLeast(3)`
    - `atMost(6)`



- Primeri
  - `unit-testing/unittesting.mock.SpyTest`

Testiranje privatnih metoda

- Privatnu metodu nije moguće pozvati direktno iz testa
- Moguća rešenja

1. Ne testirati
   - Nije dobro jer je važno taj kod izolovano testirati
2. Promeniti modifikator pristupa metodi
   - Nije dobro jer je metoda sa razlogom privatna i ne treba menjati logiku enkapsulacije zbog testova
3. Ugraditi test klasu unutar klase koja se testira
   - Nije dobro, jer time nije razdvojen kod za testiranje od produkcionog koda
4. Korišćenje refleksije
   - Jedino prihvatljivo rešenje
   - Refleksijom se može pristupiti i privatnim metodama